

Финальный проект

Что нужно сделать

- Реализовать сервис, который **асинхронно** скачивает файлы по переданному набору URL (`POST /downloads`), хранит файлы и отдает их (`GET /downloads/{id}` , `GET /downloads/{id}/files/{file_id}`).
- Хранилище файлов сделайте абстрактным repository с помощью интерфейса. **Для проекта храните файлы просто в памяти**. А в реальности это могут быть MinIO или другие S3 хранилища.
- **Для проекта реальную базу данных можно не использовать**. Можно хранить ваши модельки в памяти. Сделайте абстракцию repository с помощью интерфейса. В реальности будут запросы в PostgreSQL или другую БД.
- Скачивание файлов должно происходить **в фоне**. Т.е. вы запускаете горутини, которые:
 - **параллельно** в фоне скачивают файлы,
 - а после завершения работы обновляют статус заявки и id файлов,
 - если хотя бы один файл не скачался, то все равно продолжать скачивание остальных файлов,
- Таймаут на загрузку файлов из заявки берется из поля `timeout` тела запроса. Таймаут на всю заявку, а не на каждый файл. Т.е. что успели скачать, то успели. Спустя одну минуту мы обязаны:
 - обновить статус заявки на `DONE`,
 - прекратить загрузку того, что не успели загрузить.

API

- Ручка `POST /downloads` должна отдавать ответ **сразу же**. Т.е. вы создали заявку на скачивание в базе. Вернули ID и взяли ее в работу в фоне.
- Ручка `GET /downloads/{id}` позволяет проверить статус заявки на скачивание по ID.
- Ручка `GET /downloads/{id}/files/{file_id}` позволяет скачать файл. Сырые байты без base64.

```
// Создает заявку на скачивание набора файлов
```

```
1. POST /downloads
```

```
Req body:
```

```
{
  "files": [
    {"url": "https://google.com"}, // скачать html'ку (обычно это index.html)
    {"url": "https://somehost.com/test.pdf"} // скачать какой-нибудь pdf
  ],
  "timeout": "60s"
}
```

```
Resp body:
```

```
{
  "id": 12, // ID заявки, по которому можно опрашивать статус скачивания файлов
  "status": "PROCESS" // начальный статус заявки
}
```

```

}

// Просмотр заявки на скачивание набора файлов
2. GET /downloads/{id}
Resp body (happy path):
{
  "id": 12,
  "status": "DONE", // конечный статус заявки
  "files": [
    {"url": "https://google.com", "file_id": 79}, // ID файла для ручки
    {"url": "https://bing.com", "file_id": 80}
  ]
}
Resp body (error):
{
  "id": 12,
  "status": "DONE",
  "files": [
    {"url": "https://google.com", "error": {"code": "TIMEOUT"}}, // если файл
    {"url": "https://bing.com", "file_id": 80}
  ]
}

// Скачать конкретный файл из набора скачанных файлов
3. GET /downloads/{id}/files/{file_id}
Resp: стрим содержимого файла (Content-Type application/octet-stream)

```

Что ожидается увидеть и зачем

- **Чистая архитектура:** разделение ответственностей на transport, adapters, domain, usecases (или как вы считаете правильным).
 - Зависимости должны быть направлены правильным образом (например, domain не должен зависеть от transport).
 - Ответственность должна быть правильно разделена (например, тело HTTP запроса не должно протекать в domain, а модели из домена не должны протекать в transport).
- **Интерфейсы:** использование интерфейсов для абстрагирования.
 - Dependency inversion с помощью интерфейсов.
 - Interface segregation principle с помощью интерфейсов.
- **Context:** аккуратная работа с контекстами.
 - Пропагация пришедшего контекста из ручки во все вызовы внутри реализации ручки.
 - Обязательно таймауты на походы во внешку.
- **Goroutine:** аккуратная работа с горутинами и **обязательно ограничение срока их жизни с помощью контекста.**
 - Использование пакета sync для синхронизации доступа.
 - Использование context с таймаутом для ограничения срока жизни горутин.
 - Использование отмены контекста (например, с помощью ErrGroup), если мы понимаем, что работы горутин уже не нужна.

- Ограничение одновременного количества работающих горутин (например, с помощью ErrGroup) или семафоров.
- **Graceful shutdown**: после получения сигнала от ОС сервис должен за 1 минуту завершить работу или выбросить панику.
 - Нужно уметь ловить сигналы ОС.
 - Завершать работу приложения с помощью отмены контекста.
- **Middleware** для HTTP сервера:
 - Panic recovery.
 - Request ID - добавить request ID в контекст (сгенерировать рандомный uuid, если его не было).
- **Тесты**:
 - Юзкейсы (или даже HTTP ручки) покрыты тестами: 1 успешный ответ, 1 ответ с ошибкой.
 - Зависимости замоканы. Например, с помощью Gomock или любого другого инструменты.
- **Ошибки**:
 - HTTP коды ошибок и тела ответов адекватные:
 - если произошла **бизнесовая** ошибка (например, слишком большой файл), то отдаем **2xx с причиной**, а не просто 500,
 - если произошла **клиентская** ошибка (например, с фронта прилетел кривой запрос), то отдаем **4xx с причиной**, а не просто 500,
 - если произошла серверная ошибка, то тогда можно отдать 500.
 - В коде создаются ошибки: с помощью `fmt.Errorf`, `errors.New`, реализация интерфейса `error` и проверяются с помощью `errors.Is`, `errors.As`.

Опционально задачи на звездочку

- **Контракт**:
 - *ЛИБО*: подключите swaggo и напишите аннотации к HTTP ручкам, сгенерируйте OpenAPI контракт.
 - *ЛИБО на **две звездочки***: напишите контракт OpenAPI, по нему сгенерируйте код сервера и реализуйте его.
 - *ЛИБО на **две звездочки***: напишите Protobuf контракт, по нему сгенерируйте код gRPC сервера и реализуйте его.
- **Temporal**: добавьте добивку для пишущих операций.